

Datos, costes y registro de la búsqueda

AlgoProject — manual de uso. Generado el 2026-09-26 con `tools/manual.py`.

1. 13. La librería de barras — un solo dato guardado, todos los timeframes gratis
2. 25. Actualizar los datos — el botón "Update all", automatizado y con red
3. 24. Costes por activo — qué le cobras a cada mercado, y en qué unidad
4. 43. El libro mayor de la búsqueda — cuántas cosas se han probado de verdad
5. 12. Catálogo de rendimiento — qué cuesta cada parte del proyecto, y si hoy cuesta más que ayer

13. La librería de barras — un solo dato guardado, todos los timeframes gratis

Qué pregunta responde

¿De dónde saca Python los precios? De aquí. La librería guarda **las barras de un minuto** de cada mercado que usas, y nada más. Cuando un análisis pide M30, H1, H4 o diario, se calculan en el momento a partir del minuto y se quedan guardados para la siguiente vez.

Guardar solo el minuto no es un atajo: está comprobado que **el M30 calculado en Python es idéntico, decimal a decimal, al M30 que exporta SQX**. Medido el 21-09-2026 sobre los 7.708.823 minutos de oro: mismas 274.832 barras, mismo índice, diferencia máxima 0,000000 en apertura, máximo, mínimo y cierre. Así que tener los dos guardados sería tener el mismo dato dos veces.

Cuándo lo usas, y cuándo no

Lo usas cuando empiezas a trabajar con un mercado nuevo, y cuando actualizas la data de un mercado dentro de SQX. En los dos casos es el mismo comando y él solo decide qué hay que bajar.

No lo usas para nada más. Los análisis no lo llaman: ellos piden barras y la librería responde. Tampoco sirve para datos de tick — SQX tiene feeds de tick y el comando los ignora a propósito, porque un tick no es una barra y aquí no se remuestra nada desde ahí.

Antes de empezar

- **El worker tiene que poder arrancar.** El comando lo levanta él solo y lo deja parado al acabar. No toca el master, así que **puedes lanzarlo con la GUI de SQX abierta**.
- **El mercado tiene que estar declarado** en `assets/_markets.yaml`, o ya estar en la librería. Si añades un activo nuevo ahí, el siguiente `syncBars` lo baja sin que hagas nada más.
- No hace falta que exportes nada antes. Las barras son un hecho del mercado, no de un backtest.

Cómo se ejecuta

Primero mira qué falta, que no cuesta nada:

```
python3 -m sqx.export.syncBars --check
```

```
$ python3 -m sqx.export.syncBars --check
ok BRENTCMDUSD_ftmo      library 4,105,534  SQX 4,105,534
ok XAGUSD_DukasM1_Infinox library 7,034,806  SQX 7,034,806
ok XAUUSD_DukasM1_Infinox library 7,708,823  SQX 7,708,823
0 feed(s) to pull
```

Y cuando quieras que lo baje de verdad, el mismo comando sin `--check`:

```
python3 -m sqx.export.syncBars
```

flag	obligatorio	qué hace
<code>--check</code>	no	dice qué bajaría y no baja nada. Sin él, baja

No hay flags de fechas a propósito: **cada mercado se baja sobre su propio rango completo**, el que SQX declara para él. Si le pusieras una ventana fija, el mercado que tuviera datos más allá de esa fecha se marcaría como desactualizado para siempre.

Cuánto tarda: unos 35 segundos por mercado. Los tres de oro juntos, menos de dos minutos.

Qué produce

ruta	qué es
~/Desktop/AlgoData/bars/<feed>/M1.parquet	las barras de un minuto. 128 MB el oro, 107 el plata, 65 el Brent
~/Desktop/AlgoData/bars/manifest.json	qué tiene la librería: barras, rango y la huella de cada mercado
~/Desktop/AlgoData/barsDerived/<feed>/<TF>-<huella>.parquet	los timeframes ya calculados

La huella en el nombre del archivo es lo que hace que no tengas que acordarte de nada: si actualizas la data de un mercado en SQX y vuelves a sincronizar, la huella cambia y **los timeframes viejos dejan de usarse solos**. No se sirven barras antiguas por error nunca.

Cómo se lee el resultado

En `--check`, una línea por mercado:

- **ok** — la librería tiene exactamente las barras que tiene SQX. No hay nada que hacer.
- **PULL** — falta el mercado, o SQX tiene más barras que tú. Se va a bajar entero.
- **??** — el mercado está declarado en `assets/_markets.yaml` pero SQX no lo tiene como feed de minuto. Es un nombre mal escrito, casi siempre.

La comparación es **el número de barras**, no la fecha. Eso es lo que hace que detecte también un relleno de huecos o una corrección en medio del histórico, y no solo que haya crecido por el final.

Y lo que cuesta pedir un timeframe, medido de verdad sobre el oro:

```
$ python3 -c 'from core import barstore; barstore.read("XAUUSD_DukasM1_Infinox", TF)'  
timeframe    barras    1a vez    cacheado  
M30          274,832   1.32s     0.031s  
H1           137,785   1.15s     0.025s  
H4           35,299    1.01s     0.012s  
D1           5,912     0.99s     0.005s
```

La primera vez de cada timeframe cuesta **un segundo largo**: ahí está leyendo los 7,7 millones de minutos. A partir de ahí, centésimas. Un informe que analiza 757 estrategias no nota ese segundo; el panel interactivo tampoco, porque solo lo paga la primera vez que lo abres tras actualizar data.

Un ejemplo completo

Actualizaste el Brent en SQX y quieres que Python lo vea:

```
python3 -m sqx.export.sync_bars
```

```

$ python3 -m sqx.export.syncBars
PULL BRENTCMDUSD_ftmo          library 4,018,086   SQX 4,105,534
ok   XAGUSD_DukasM1_Infinox     library 7,034,806   SQX 7,034,806
ok   XAUUSD_DukasM1_Infinox     library 7,708,823   SQX 7,708,823
BRENTCMDUSD_ftmo               4,105,534 bars    2013-01-01 → 2026-09-07  65 MB
1 feed(s) refreshed; library holds 3

```

Lee lo que pasó: el oro y la plata estaban al día y **no se tocaron**. El Brent tenía 4.018.086 barras en la librería contra 4.105.534 en SQX, así que se bajó entero — hasta el 7 de septiembre, que es hasta donde llega su data. Los timeframes del Brent que hubiera calculados quedaron huérfanos en ese momento y se recalculan la próxima vez que alguien los pida.

Qué NO te dice

- **No te dice si la data de SQX es buena.** Copia lo que SQX tenga. Si un feed tiene huecos, la librería tendrá los mismos huecos.
- **No te avisa de que tu data está vieja.** Solo compara tu librería contra SQX. Si en SQX llevas ocho meses sin actualizar un mercado, los dos dirán `ok` tan contentos. Eso se mira en SQX. *(A 21-09-2026 el oro y la plata llegan hasta el 16-01-2026 y el Brent hasta el 07-09-2026: los metales llevan ocho meses sin actualizar en SQX, y esto no te lo iba a decir el comando.)*
- **exportBars.py ya no existe** (retirado el 23-09-2026). Un timeframe suelto se pide a `core.barstore.bars(feed, TF)`, que lo calcula desde el minuto en un instante y lo cachea.
- **El volumen no es exacto al de SQX.** Calculado desde el minuto, difiere en 13 barras de 274.832 y como mucho en 2 unidades — redondeo de SQX al agregar. Los precios sí son exactos. Si algún día haces algo que dependa del volumen al detalle, tenlo presente.

Si algo falla

- **KeyError: '<feed>'** al pedir barras — ese mercado no está en la librería. Corre `syncBars --check` y mira si sale como `??`: entonces el nombre en `assets/_markets.yaml` no coincide con el de SQX.
- **El comando se queda colgado al arrancar** — el worker no levantó. `bin/sqx-worker.sh stop` y vuelve a lanzarlo.
- **Un mercado sale PULL una y otra vez aunque acabe de bajarlo** — eso sería un fallo de verdad y hay que mirarlo, no repetirlo. Pasó una vez, con una ventana de fechas fija que recortaba el feed; por eso ya no hay flags de fechas.

25. Actualizar los datos — el botón "Update all", automatizado y con red

Qué pregunta responde

Ninguna: esto no analiza nada. Hace una cosa y la hace con cuidado — descarga histórico nuevo en el maestro, igual que el botón **"Update all"** de la GUI, y después deja `assets/_policy.yaml` diciendo la verdad sobre hasta dónde llega cada feed.

Las dos mitades van juntas a propósito. Actualizar sin refrescar las fechas deja el proyecto creyendo que tu histórico acaba donde acababa hace meses.

Cuándo lo usas, y cuándo no

Lo usas cuando quieras la descarga desatendida — de madrugada, sin nadie delante — o cuando prefieras que el refresco de fechas vaya encadenado y no se te olvide.

No lo uses si sólo quieres refrescar las fechas sin descargar nada. Para eso está `python3 -m core.assets --dataranges`, que pregunta al conductor, no toca el maestro y **es seguro con tu GUI abierta**.

Y no lo uses con la GUI del maestro levantada, porque no te va a dejar.

Antes de empezar

Cierra StrategyQuant en el maestro. No hay forma de saltárselo y es deliberado. Si está abierto:

```
SQLX está corriendo desde /home/sergioguslw/Desktop/SQLX (PID 234724, 235060, 235064, ...).  
Ciérralo tú y vuelve a lanzarlo: la regla dura 2 prohíbe sqcli en el maestro  
con la GUI levantada, y este comando acaba en un sync que borra .sqx de disco.
```

Sale con 1 y no toca nada. **Nunca mata el proceso:** lo cierras tú.

Cómo se ejecuta

```
python3 -m sqx.data.update          # ENSAYO: te dice qué haría y no hace nada  
python3 -m sqx.data.update --apply  # descarga de verdad  
python3 -m sqx.data.update --apply --symbol XAUUSD_DukasM1_Infinox
```

flag	obligatorio	qué hace
<code>--apply</code>	no	sin él es un ensayo. Es lo único que descarga
<code>--symbol</code>	no	un solo símbolo. Por defecto, todos los que el maestro tenga configurados

El ensayo imprime el comando exacto que lanzaría y cuántas estrategias hay en juego:

```
7546 .sqx en disco bajo /home/sergioguslw/Desktop/SQLX/user/projects  
ENSAYO. Lanzaría: ./sqcli -data action=update (cwd /home/sergioguslw/Desktop/SQLX)  
Añade --apply para hacerlo de verdad.
```

Por qué cuenta los `.sqx` — esto es lo importante

Toda ejecución de `sqcli` termina así, la mires o no:

Y la **regla dura 1** dice qué significa eso: *cada sync de SQX borra los `.sqx` de disco que el databank no tiene en memoria*. En tu maestro eso son **7.546 estrategias y 3,4 GB** expuestos cada vez que se lanza cualquier comando.

Por eso el comando fotografía `user/projects` entero antes y después, guarda las dos fotos en `AlgoData/backups/data-update/` y compara:

```
foto guardada en .../AlgoData/backups/data-update/2026-09-22T213045Z.json
...
regla 1 ok: los 7546 .sqx siguen en disco
```

Si algo desapareciera lo dice con nombre y tamaño, y tienes la foto para saber exactamente qué era. Un informe vacío es el único resultado aceptable.

Por qué en el maestro y no en un worker

Porque una descarga hecha en un worker **se pierde**. `bin/sqx-worker.sh` hace esto en cada arranque:

```
rsync -a --exclude='History/' "$MASTER/user/data/" "$WORKER/user/data/"
```

Maestro → worker. El `History` es un enlace compartido, así que los ficheros de barras nuevos caerían bien, pero las tres bases H2 que un backtest lee de verdad son copias por instalación: la copia fresca del worker la machaca la vieja del maestro en el siguiente arranque, y tu GUI sigue construyendo con barras antiguas. Silencioso. Por eso la descarga va donde va.

Qué hace al terminar

Encadena el refresco de fechas él solo, así que no hay un segundo paso que olvidar:

```
XAUUSD: data: {from: 2003-05-05, to: 2026-01-16} # ..., 7.708.823 barras M1
→ data: {from: 2003-05-05, to: 2026-09-22} # ..., 7.949.285 barras M1
```

Qué NO hace

- **No cierra SQX por ti**, ni lo mata. Se niega y te lo dice.
- **No toca ningún proyecto**. Sólo descarga datos y reescribe las fechas de `_policy.yaml`.
- **No decide ventanas**. Que un tramo `oos2` llegue más allá de los datos es un aviso del preflight, no algo que esto arregle. Ver `03-datos-costes-y-registro.pdf > 24-costes`.

Ojo

❑ **Probada de punta a punta el 2026-09-25**. Sin `--symbol` actualizó los **67 símbolos** que tiene el maestro en **16 minutos**, y los 7.546 `.sqx` siguieron en disco. Los diecisiete activos pasaron de acabar el `2026-09-22` a acabar el `2026-09-25`.

⚠ **Algún día puede fallar por límite de descargas**. Aquella vez, dos días de `MSFT_DukasM1_ICMarkets` dieron `ERROR: download failed. Error code: 429`: el servidor de datos cortó por exceso de peticiones. No hace falta arreglar nada; la siguiente ejecución los baja. Búscalo así en la salida:

```
grep 'ERROR: download failed' <salida>
```

Haz igualmente una copia de `user/projects` antes (regla dura 1): la foto que guarda el comando es un inventario, no una copia.

Automático, cada sábado

Desde el 2026-09-25 cron lo lanza **los sábados a las 03:00** con `bin/weekly-data-update.sh`, sin modelo de por medio. Hace tres cosas en orden y, si una falla, no hace la siguiente:

1. Copia `user/projects` del maestro entero (unos 4 GB) en `AlgoData/snapshots/weekly-data-update-AAAA-MM-DD`, y comprueba que la copia tiene los mismos `.sqx`. Guarda las tres últimas copias y borra las anteriores, solo las suyas.
2. Lanza `python3 -m sqx.data.update --apply`. **Si tu GUI del maestro está abierta, no hace nada**, ni la copia.
3. Apunta los días que el servidor de datos rechazó (el 429 de arriba).

Lo que pasó lo lees en `AlgoData/logs/weekly-data-update.log`, ya sin las miles de líneas de ruido de `sqcli`. Para probarlo sin lanzar nada:

```
bin/weekly-data-update.sh --dry-run
```

Si un worker arranca durante la descarga, copia las bases de datos del maestro a medio escribir. Por eso a esa hora no debería haber nada arrancando.

Dónde está el detalle

- `sqx/data/README.md` — el porqué del maestro y del rsync.
- `knowhow/costs/refreshing-sqx-costs.md` — la referencia de verbos y el flujo de datos entre instalaciones.
- `03-datos-costes-y-registro.pdf > 24-costes` — el refresco de fechas por separado, sin descargar nada.

24. Costes por activo — qué le cobras a cada mercado, y en qué unidad

Qué pregunta responde

Antes de construir o retestear nada hay que saber qué paga esa estrategia por operar: el spread, la comisión y el swap. SQX trae sus propios números y **casi nunca son los que te cobra tu bróker**, así que esa diferencia se escribe una vez en `assets/` y se consulta siempre. Esta página dice cómo está organizado eso, qué unidad lleva cada campo, y por qué un activo de forex se rellena distinto que el oro.

También contesta la otra mitad: en qué otros mercados hay que comprobar que la estrategia sigue funcionando, que es `assets/_markets.yaml`.

Cuándo lo usas, y cuándo no

Lo usas antes de tocar cualquier proyecto, tarea o plantilla de un símbolo. Es la regla dura 5 del `CLAUDE.md` y no es opcional: si el activo tiene un coste sin decidir, el comando falla y el trabajo se para hasta que tú des la cifra.

No lo usas para saber qué pagó de verdad un backtest que ya corrió. Eso no se supone, se recupera de las propias operaciones (`gross - P/L`), y lo hace `portfolio/common/monteCarlo/inputs/costs.py`. `assets/` dice lo que hay que aplicar de aquí en adelante, no lo que se aplicó.

Antes de empezar

Nada. No hace falta SQX abierto ni datos exportados: son ficheros de texto y un comando que los lee.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD          # el preflight de un activo
python3 -m core.assets --index          # regenera assets/INDEX.md
python3 -m core.assets --dataranges     # relee de SQX hasta dónde llega cada feed
```

El tercero es el que usas **después de darle a "Update all" en el maestro**: pregunta a SQX qué histórico tiene ahora de cada feed y reescribe el `data:` de los 17 en `_policy.yaml`. Sólo imprime lo que se movió:

```
XAUUSD: data: {from: 2003-05-05, to: 2026-01-16}  # ..., 7.708.823 barras M1
  → data: {from: 2003-05-05, to: 2026-09-22}  # ..., 7.949.285 barras M1
```

Pregunta al **conductor**, no al maestro, así que puedes lanzarlo con tu GUI abierta sin ningún riesgo. Ve los datos del maestro igualmente porque `bin/sqx-worker.sh` copia su `user/data` al worker en cada arranque, y el `History` es un enlace al del maestro.

Salida real del primero:


```
# XAUUSD – clase `no_forex`, overrides a aplicar

SQX symbol: XAUUSD_DukasM1_Infinox    verificado: 2026-09-03

- **spread_is**: usar `10.0` points (SQX lleva hoy `10`) – PROVISIONAL 2026-09-21 – default de SQX...
- **spread_oos**: usar `10.0` points (SQX lleva hoy `10`) – PROVISIONAL 2026-09-21 – default de SQX...
- **commission**: usar `0.00229` pct_of_notional (SQX lleva hoy `{'method': 'SizeBased', 'value': 8}`)...
- **swap_long**: usar `SIN DECIDIR` pct_annual (SQX lleva hoy `{'type': 'points', 'value': -73.42}`)
- **swap_short**: usar `SIN DECIDIR` pct_annual (SQX lleva hoy `{'type': 'points', 'value': 38.76}`)
- **segmento build**: 2008-01-01 a 2017-12-31 inclusive (dateFrom 1199145600000, dateTo 1514764800000)
- **segmento oos1**: 2018-01-01 a 2022-12-31 inclusive (dateFrom 1514764800000, dateTo 1672531200000)
- **segmento oos2**: 2023-01-01 a 2026-12-31 inclusive (...) ⚠ RESERVADO para WFC, WFM
- **retest family**: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
- **retest structural**: (vacío)
```

Los tres códigos de salida, que son el mecanismo entero:

sale con	qué significa	qué haces
0	todo decidido	sigues, diciendo en voz alta qué valores aplicas
2	algún coste obligatorio sigue en <code>use: null</code>	paras y preguntas. No eliges tú un número
3	el fichero no cuadra con su clase	lo arreglas: te dice exactamente qué campo falta o sobra

Cómo está organizado

Tres ficheros compartidos arriba y los diecisiete de instrumento en `symbols/`, para que la carpeta se vea de un vistazo:

fichero	qué guarda	cuántos hay
<code>_policy.yaml</code>	qué es cada tramo IS/OOS (una vez) y dónde empieza y acaba en cada activo , con la fecha de datos de SQX al lado	uno, para todos
<code>_classes.yaml</code>	los dos esquemas de coste: qué campos, en qué unidad, en qué ajuste de SQX	uno, para todos
<code>_markets.yaml</code>	el universo de retest: por activo main, mercados <code>family</code> y <code>structural</code>	uno, para todos
<code>symbols/<SÍMBOLO>.yaml</code>	lo propio del instrumento: sus costes y sus rangos de MC Retest	uno por activo, en su subcarpeta

La idea es que abras un fichero de activo y **no veas nada repetido**. Los segmentos no están diecisiete veces, están una. Lo que quedó dentro es lo que de verdad cambia de un mercado a otro.

Los tramos IS/OOS: la plantilla se comparte, las fechas son de cada activo

Los históricos no empiezan a la vez. El oro tiene M1 desde el 2003-05-05 y tú construyes desde 2008; el DAX40 no tiene nada antes del 2013-09-30, así que una ventana que empiece en 2008 se quedaría corta **sin avisar**. Por eso `_policy.yaml` parte el asunto en dos:

- `segments_default` — **qué es** cada tramo: su papel y qué spread usa. Una sola copia.
- `segments` — **dónde empieza y acaba en cada activo**, con `data` al lado. Los 17 juntos, para poder compararlos y ajustarlos de una vez en lugar de abrir diecisiete ficheros.

```
segments:
  XAUUSD:
    data: {from: 2003-05-05, to: 2026-01-16} # leído de SQX, no se edita a mano
    build: {from: 2008, to: 2017}
    oos1: {from: 2018, to: 2022}
    oos2: {from: 2023, to: 2026}
  DAX40:
    data: {from: 2013-09-30, to: 2026-01-16}
    build: {from: null, to: null} # sin decidir
```

Un extremo puede ser **un año o una fecha**. El año significa el año entero, ambos extremos incluidos: `from: 2008, to: 2017` es del 1 de enero de 2008 al 31 de diciembre de 2017 completo. La fecha significa ese día, también incluido, y es como se corta un tramo a mitad de año: `to: 2026-08-30` llega hasta el último instante del 30 de agosto.

Las ventanas de forex y metales, decididas el 2026-09-22:

tramo	desde	hasta
build	2008-01-01	2017-12-31
oos1	2018-01-01	2022-12-31
oos2	2023-01-01	2026-08-30

Los seis índices siguen sin ventana a propósito: sus históricos empiezan en 2011-2013 y una ventana de 2008 no cabría — el comando la rechazaría.

`data` lo lee el proyecto de SQX con `bin/sqx-worker.sh run -symbol action=list` y está ahí justo para que veas si la ventana que pones cabe. **Si no cabe, el comando falla:**

```
ESQUEMA ROTO en assets/symbols/DAX40.yaml: el tramo `build` empieza en 2008 y los datos de
`DAX40_DukasM1_Infinox` empiezan en 2013-09-30
```

→ exit 3. Eso es fatal: esa ventana no se puede llenar nunca. En cambio, que un tramo **acabe después** del último dato es sólo un aviso, porque es el estado normal del tramo más reciente mientras no sincronizas:

```
AVISO: el tramo `oos2` llega a 2026-08-30 y los datos de `XAUUSD_DukasM1_Infinox`
acaban en 2026-01-16
```

Y si las dejas en `null`, nadie se inventa una ventana: `window()` se niega y el comando avisa.

Lo que tiene SQX hoy, leído el 2026-09-22:

activo	datos desde	activo	datos desde
EURUSD, GBPUSD, USDCHF, USDJPY, XAUUSD	2003-05-05	NIKKEI225	2011-09-19
AUDUSD, EURJPY, GBPJPY, USDCAD	2003-08-04	USA500, USATEC	2012-01-19
AUDJPY	2003-12-01	DAX40, DJ30	2013-09-30
CADJPY	2004-10-25	SP500ft	no existe en SQX

Las dos clases, que es lo que decide cómo se rellena

	forex	no_forex (índices y metales)
spread	spread , un valor en puntos	spread_is y spread_oos , en puntos
comisión	commission , \$ por lote (SizeBased)	commission , % del nocional (PercentageBased)
slippage	slippage , en puntos	igual
swap	swap_long / swap_short , puntos por noche	swap_long / swap_short , % ANUAL

Por qué dos spreads fuera de forex. El precio de estos activos se multiplica a lo largo del histórico: el oro va de 800 a 3500. Un spread fijo en puntos cuesta siempre lo mismo en dólares, así que en porcentaje cuesta **el triple en 2010 que hoy** — y eso mete una pendiente dentro de la misma muestra que ve el generador. Declarando uno para el tramo de construcción y otro para los dos fuera de muestra, esa pendiente desaparece. Se puede hacer porque **SQX guarda los costes dentro de cada tarea**, no una vez por proyecto: la tarea de build lleva un `defaultSpread` y las de retest otro.

Por qué el % fuera de forex. Un porcentaje escala con el precio él solo, que es justo lo que le falta a un número fijo en puntos.

Los rangos del MC Retest

La tarea **MC Retest** vuelve a correr el backtest entero contra una entrada perturbada, y dos de sus métodos sortean un valor dentro de un rango: el spread y el slippage. Ese rango lo pones tú, por activo, al final de su fichero:

```
mc_retest:
  spread:
    min: 5                # puntos
    max: 12
    sqx_now: [{min: 1, max: 5}, {min: 5, max: 12}]  # varias tareas, rangos distintos
  slippage:
    min: 0
    max: 10
    sqx_now: [{min: 0, max: 5}, {min: 0, max: 10}]
```

Los dos van en puntos absolutos, no en múltiplos del spread real. Eso es lo que obliga a que sean por activo: un rango de 1 a 5 puntos es razonable en EURUSD y no significa nada en el NIKKEI225.

Si los dejas sin decidir **no se bloquea nada**: el comando avisa y sale con 0, porque un rango sin fijar sólo deja sin interpretar esa tarea concreta del MC Retest, no el resto del trabajo.

AVISO: los rangos MC Retest spread, slippage no están decididos. No bloquea, pero esa tarea del MC Retest no es interpretable hasta que el dueño los fije.

△ Mirando el maestro el 2026-09-22, **casi todos los instrumentos llevan el rango de fábrica de SQX** —spread 1,0–5,0 y slippage 0,0–5,0— que es idéntico en todos los mercados. En el NIKKEI225, cuyo spread real son 1100 puntos, eso perturba entre 1 y 5: unas 200 veces más barato que el propio backtest. Donde sí los has puesto a mano están a la escala del activo: el oro lleva spread 5–12 contra un spread real de 10, y el NIKKEI225 lleva 80–200 y 120–400 en sus otras tareas.

(`RandomizeMinDistance` tiene la misma forma y rango de fábrica 0,0–10,0. No está en `assets/` porque pediste estos dos.)

Las dos trampas, que cuestan dinero de verdad

⚠ **El swap en % es ANUAL, no por noche.** SQX divide entre 100 y entre 360 antes de aplicarlo. Si escribes ahí el porcentaje de una noche, te equivocas por 360 veces. Para pasar un swap que hoy está en puntos:

```
pct_anual = puntos × tick_size × 36000 / precio_de_referencia
```

Los `-73.42` puntos del oro, con tick 0.01 y precio 3500, son **7,55 % anual**; los `+38.76` del corto son **3,99 %**. Comprobado: por las dos vías, una noche de un lote cuesta 73,42 \$ y 38,76 \$.

⚠ **La comisión en % puede estar cobrándose por pata o por operación, y no está verificado.** El código de SQX sólo cobra al abrir, pero lo medido en los exports con `SizeBased 8` son 16 \$ ida y vuelta. Es un factor de 2 sobre toda la comisión de los activos no-forex. Antes de dar por buena una cifra, hay que comprobarlo con un backtest y el residual `gross - P/L`.

Los dos tipos de mercado adicional

`_markets.yaml` separa en dos, y la diferencia importa porque responden a preguntas distintas:

- **family** — mismo motor económico que el activo principal. Oro y plata, por ejemplo. Es la prueba fácil: **pasarla no demuestra gran cosa**, porque puede que los dos mercados sean la misma operación con otro nombre. Fallarla sí dice algo.
- **structural** — parecidos en estructura (volatilidad, horario, ruido) pero **sin ningún motor compartido**. Es la prueba dura: si el edge transfiere aquí, es que la lógica capta algo del movimiento del precio y no una exposición a un factor macro.

La lista se cierra antes de mirar ningún resultado. Elegir los mercados después de ver dónde funciona convierte la prueba en una selección, y sus p-valores en decoración.

Cómo se rellena un valor

Abres `assets/<SÍMBOLO>.yaml` y en el bloque del campo pones el número en `use:` y el motivo en `why:`. `sqx_now` no se toca: es lo que SQX lleva hoy, y está ahí para que veas de qué partes — ojo, **muchas veces en otra unidad** que la que va en `use`.

```
costs:
  spread_is:
    use: 14                # points - tramo `build`
    sqx_now: 10            # points, el unico que hay hoy
    why: "Infinox, media de la sesión de Londres 2008-2017, confirmado 2026-XX-XX"
```

Sólo el **spread y la comisión bloquean**. El slippage y los swaps no, y los dos llevan ya una cifra PROVISIONAL: el slippage es **la mitad del spread** de cada instrumento.

No es una medición y no puede serlo — un backtest aplica el slippage que le das, así que de un export no se recupera nunca el real. Es una convención, y se eligió ésa porque es la única que aguanta entre clases: un punto vale `tick_size × point_value`, que va de 0,0065 \$ en el Nikkei a 11,53 \$ en USDCHF, así que 2,5 puntos planos serían 0,02 \$ en uno y 28,82 \$ en el otro. La mitad del spread deja el coste entre 0,33 \$ y 5,00 \$ por lote y pata, y en el oro cae en 5 puntos: el punto medio del rango 0-10 que tu propio MC Retest ya sortea.

Si escribes `PROVISIONAL` en el `why`, el valor **decide pero no bloquea**: el comando sale con 0 y todo informe construido encima queda marcado, para que dentro de seis meses nadie confunda un número de relleno con una cifra pactada con el bróker.

Cómo se sabe que está bien

`python3 -m core.assets <SÍMBOLO>` sale con 0 y el índice lo marca:

```
python3 -m core.assets --index && head -8 assets/INDEX.md
```

□ decidido, ▲ provisional decidido con un número de relleno, — bloqueado.

Qué NO hace

- **No escribe en SQX.** `core.assets.sqx_settings()` devuelve lo que una tarea tendría que llevar, pero meterlo en el proyecto es otro trabajo. Hoy hay que hacerlo a mano en la GUI.
- **No adivina.** Si no hay cifra pactada, hay `null` y el trabajo se para. Un fichero con valores inventados es peor que no tener fichero: parece decidido.
- **No sabe qué te cobró un backtest ya corrido.** Eso se recupera de las operaciones.

Dónde está el detalle

- `assets/RULES.md` — la regla y el esquema, para una sesión de Claude.
- `knowhow/costs/` — qué puede cobrar SQX y en qué unidad, con las fórmulas decompiladas.
- `assets/_classes.yaml` — los dos esquemas, uno al lado del otro.

43. El libro mayor de la búsqueda — cuántas cosas se han probado de verdad

Qué pregunta responde

Cada paso de la cadena cuenta su propio embudo y se le olvida. Cuando llegas al paso 17 con tres supervivientes, **nadie sabe si salieron de diez mil o de cincuenta** — y esa diferencia no es contabilidad, es el resultado.

Este módulo es el libro que no se olvida. Escribe **una línea por búsqueda** —una construcción, un retest, una criba de Python, un umbral aplicado— durante toda la vida de un estudio, y con eso contesta tres cosas que hoy nadie puede contestar:

1. **El embudo entero**: de cuántas a cuántas, paso a paso, con los umbrales de cada corte.
2. **Cuánta historia se ha gastado**: qué tramos ha mirado alguien, cuántas veces, y si el `oos2` sigue siendo virgen de verdad.
3. **Cuántas cosas se han probado en total**, que es el número que el Sharpe desinflado necesita y que hoy recibe subestimado — porque se le da la dispersión de un lote de variantes en vez de la de todo el estudio.

Y hace cumplir dos reglas que hasta ahora dependían de la buena voluntad de quien corriera la cadena: **nadie mira el `oos2` si no es el paso 17 o el 19, y el paso 20 no se lee hasta que 17, 18 y 19 estén los tres hechos**. No avisa: se niega.

Cuándo lo usas, y cuándo no

Lo usas siempre. Un estudio sin ledger no puede decir lo que vale su superviviente.

No lo usas para decidir nada por ti: no puntúa estrategias ni las descarta. Cuenta.

Y **no edites `ledger/thresholds.yaml` para que algo pase**. Un umbral movido después de ver los resultados es la manera más barata de sobreajustar la cadena entera.

Antes de empezar

Nada. El fichero de un estudio se crea solo la primera vez que se le escribe, en

```
~/Desktop/AlgoData/ledger/<estudio>.jsonl.
```

Un **estudio** es un activo + un timeframe + una plantilla o familia de plantillas: `XAUUSD_M30_DirectionalMomentum`. Es la unidad a la que se le debe la corrección por comparaciones múltiples: todas las búsquedas que han tocado ese activo con esa plantilla cuentan juntas, venga del paso que venga.

Cómo se ejecuta

Leer un estudio:

```
python3 -m ledger.report --study XAUUSD_M30_DirectionalMomentum
```

Comprobar que cada umbral llega a su módulo desde el ledger:

```
python3 -m ledger.report --check-thresholds
```

Reconstruir hacia atrás un estudio que ya corrió, desde lo que dejó en disco:

```
python3 -m ledger.backfill \
  --gate ~/Desktop/AlgoData/reports/XAU_IS00S_ejemplo/Results/2026-09-23/gate \
  --symbol XAUUSD --timeframe M30 --family DirectionalMomentum --write
```

flag	obligatorio	qué hace
--study	sí (en report)	el identificador del estudio
--check-thresholds	—	dice de cada umbral si su módulo lo lee del ledger o de una copia, y sale con error si una copia diverge
--equity + --identity	no	añade el Sharpe desinflado de una superviviente concreta
--gate	sí (en backfill)	el directorio del informe de la puerta
--write	no	sin él, backfill enseña lo que haría y no escribe

Segundos, y no toca SQX.

Qué produce

Un `.jsonl` por estudio bajo el data root: **una línea JSON por búsqueda, y nada la reescribe nunca**. Se añade al final; no hay borrado ni edición, porque un libro que se puede editar después contesta una pregunta distinta de aquella para la que se hizo.

Cómo se lee el resultado

```
XAUUSD_M30_DirectionalMomentum · 8 búsquedas ·  
/home/.../AlgoData/ledger/XAUUSD_M30_DirectionalMomentum.jsonl  
  
-- el embudo, paso a paso  
      ts  step      criterion segment  n_in  n_out  kept  
2026-09-23    8  studies/screening/gate/presencia  oos1  120   115  0.9583  
2026-09-23    8  studies/screening/gate/sanidad  oos1  115   112  0.9739  
2026-09-23    8  studies/screening/gate/estaticas  oos1  112    64  0.5714  
2026-09-23    8  studies/screening/gate/degradacion  oos1   64    45  0.7031  
2026-09-23    8  studies/screening/gate/forma  oos1   45    45  1.0000  
2026-09-23    8  studies/screening/gate/mono  oos1   45    45  1.0000  
2026-09-23    8  studies/screening/gate/familia  oos1   45    45  1.0000  
2026-09-23    8  studies/screening/gate/redundancia  oos1   45    45  1.0000  
de 120 entraron a 45 supervivientes  
  
-- qué historia se ha gastado  
segment  searches steps      first      last  
  oos1           8   [8] 2026-09-23 2026-09-23  
build  2008 → 2017  leído 0x  
oos1   2018 → 2022  leído 8x  
oos2   2023 → 2026-08-30  leído 0x  RESERVADO para WFC, WFM  
  
-- la puerta ciega del paso 20  
17 pendiente · 18 pendiente · 19 pendiente  
ledger: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19].  
Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala  
  
-- lo que se ha probado en total: N = 115 candidatos sobre 1 búsqueda(s), sigma = 0.3506 (annualised)  
benchmark con N de la última búsqueda ( 115): 0.0570 (por día)  
benchmark con N de el estudio entero ( 115): 0.0570 (por día)  
los dos coinciden: este estudio sólo tiene una búsqueda con distribución registrada
```

El embudo se lee de arriba abajo. `kept` es qué fracción sobrevivió a ese corte. Aquí las estáticas se llevaron casi la mitad (112 → 64) y la degradación otro 30 %.

El mapa de gasto es el que contesta «¿sigue siendo holdout mi holdout?». `oos1` leído 8 veces ya no es fuera de muestra en ningún sentido útil; `oos2` con 0 lecturas sí lo es.

La puerta ciega dice qué falta para poder leer el paso 20. Mientras falte uno, cualquier módulo que llame a `gate.allow_read` revienta en vez de enseñarte números.

El total probado es la razón de ser del módulo: `sigma` es la dispersión de los candidatos entre los que se eligió, y `N` cuántos fueron. Los dos alimentan el Sharpe desinflado. En este estudio los dos benchmarks coinciden porque **sólo una búsqueda registró su distribución** — el `backfill` sólo puede recuperar lo que la puerta dejó escrito, y de los pasos anteriores no queda registro. Con dos búsquedas ya se separan: medido en `tests/test_ledger.py`, al pasar de `N=1.600` a `N=2.000` el listón sube de 0,1995 a 0,2031 y el Sharpe desinflado baja de **0,6301 a 0,6000**.

⚠ **Las unidades no se mezclan.** SQX guarda Sharpes anualizados y las fórmulas de `core/significance.py` trabajan por observación. Cada línea apunta en qué unidad venían sus candidatos, y agrupar dos unidades distintas **lanza** en vez de dar un número creíble y falso.

Un ejemplo completo

La comprobación de umbrales, con salida real:

	key	declarado	lee_de	coincide	fijado_por	el
	gate.sanidad.min_trades	20.00	ledger	True	dueño	2026-09-23
	gate.degradacion.min_retention	0.00	ledger	True	dueño	2026-09-23
	gate.degradacion.max_concentration	1.00	ledger	True	dueño	2026-09-23
	gate.forma.max_dd_ratio	5.00	ledger	True	dueño	2026-09-23
	gate.mono.max_p	0.50	ledger	True	dueño	2026-09-23
	gate.familia.alpha	0.05	ledger	True	dueño	2026-09-23
	snoopingScreen.stepm.fwer	0.05	ledger	True	dueño	2026-09-25
	profitShape.max_top5	0.60	ledger	True	agente	2026-09-24
	profitShape.alpha	0.05	ledger	True	agente	2026-09-24
	entryQuality.dcr_high	0.30	ledger	True	agente	2026-09-24
	parameterCloud.rank_high	0.95	ledger	True	agente	2026-09-24
	parameterCloud.plateau_low	0.15	ledger	True	agente	2026-09-24
	parameterCloud.rho_low	0.20	ledger	True	agente	2026-09-24
	cscv.blocks	12.00	ledger	True	dueño	2026-09-23

14 umbrales declarados: 14 los lee su módulo del ledger y 0 son copias que coinciden

Desde el 2026-09-26 **los módulos leen cada umbral de aquí:** donde su `config.yaml` tenía el número ahora pone `ledger:<clave>`, y el módulo lo sustituye al leer. La columna `lee_de` dice `ledger` cuando es así y `copia` cuando un módulo todavía guarda su propio número; una copia que no coincide, o un hueco que apunta a otra clave, sale como divergencia y la orden termina con error — que es como está pensado para meterse en un guion. Con los seis módulos migrados no queda ninguna copia. **Para cambiar un umbral, cámbialo aquí,** con tu nombre y la fecha: el `config.yaml` ya no tiene el número. `--set` sigue sirviendo para probar otro valor en una corrida sin tocar nada.

Qué NO te dice

- **No sabe lo que no se registró.** Una búsqueda que no llamó al ledger no existió para él, y los pasos anteriores a que esto existiera sólo se recuperan si dejaron un artefacto por escrito. Las líneas reconstruidas van marcadas `backfill` y llevan la fecha del informe, no la de hoy.
- **No decide nada.** No puntúa, no descarta y no ordena.

Si algo falla

- `PermissionError`: el paso N no puede mirar `oos2` — **no es un fallo, es la puerta.** Si crees que ese paso sí debería poder, se arregla en `assets/_policy.yaml`, que es tuyo, y no ensanchando la comprobación.
- `ValueError`: las búsquedas mezclan unidades de `score` — alguien registró Sharpes anualizados y otro por observación en el mismo estudio. Hay que decidir cuál y rehacer el registro de esa fase.
- `KeyError`: `'<clave>: declared 0 times in thresholds.yaml'` (o 2 times) — un módulo pide un umbral que el registro no tiene, o que dos ramas añadieron dos veces. Se arregla en `ledger/thresholds.yaml`: una fila por clave. No se arregla poniendo el número en el `config.yaml`.
- `sin búsquedas registradas todavía` — el estudio no tiene fichero: o el identificador no coincide (mira `~/Desktop/AlgoData/ledger/`) o nadie ha escrito nada aún.

12. Catálogo de rendimiento — qué cuesta cada parte del proyecto, y si hoy cuesta más que ayer

Qué pregunta responde

Este proyecto es caro: miles de estrategias, cientos de miles de simulaciones, y una máquina que comparte con la GUI de StrategyQuant X. El catálogo mide **cuánto tarda y cuánta memoria pide** cada parte cara del código, guarda la medida con la fecha y el commit, y la próxima vez te dice si algo se ha vuelto más lento.

También mide el disco: cuánto ocupa cada rama de `AlgoData`, qué ficheros están duplicados, y cuánto costaría leer los mismos datos guardados en otro formato.

No es un test. No dice si el código está bien: dice lo que cuesta.

Cuándo lo usas, y cuándo no

Úsalo después de tocar cualquier módulo de análisis, antes de lanzar una corrida larga que tiene que caber en memoria, y cuando quieras saber dónde merece la pena invertir tiempo de programación.

No lo uses para comparar con la medida de otra máquina: los números son de este servidor. Y no lo uses como prueba de que un cambio es correcto — puede ser tres veces más rápido y estar mal.

No hace falta cerrar SQX. Ningún objetivo toca la instalación: todos leen ficheros ya exportados bajo `~/Desktop/AlgoData`. Pero si la GUI está construyendo estrategias, la máquina está ocupada y las medidas saldrán peores; `history.csv` guarda la carga del sistema con cada fila para que eso se vea.

Antes de empezar

Tiene que existir un export de trabajo bajo el raíz de datos. Por defecto el catálogo mide sobre:

qué	dónde	se cambia en
operaciones exportadas y sus <code>.sqx</code>	ninguna desde el 2026-09-25 (se borró <code>MC_Trades</code>): hay que elegir una exportación con <code>trades/</code> y <code>strategies/</code>	<code>sample.trades_databank</code>
batería de retest ya ingerida	<code>raw/XAUUSD/MCR_All/<fecha>/</code>	<code>sample.retest_databank</code>
velas	<code>bars/XAUUSD_DukasM1_Infinox/M30.csv</code>	<code>sample.bars_feed</code>

Siempre usa **el export más reciente** de cada uno. Si no tienes alguno, exporta primero (`01-empezar.pdf` > `00-empezar`) o cambia esas claves en `perf/config.yaml`.

Cómo se ejecuta

```
python3 -m perf.catalogue
```

```
$ python3 -m perf.catalogue --only core
midiendo en 779d2e1-dirty
  core.trades_read ...
  core.bars_read ...
  core.sqx_xml ...
  core.sqx_stats ...
target      ahora      antes      tiempo  memoria  veredicto
core.trades_read    0.17s    0.17s    1.6%   -1.4%  noisy
core.bars_read      0.41s    0.41s    0.5%   -5.3%  noisy
core.sqx_xml        0.03s    0.04s   -0.7%    0.4%  noisy
core.sqx_stats       0.05s    0.05s    0.1%    0.4%  noisy
montecarlo.stream    0.04s    0.04s   -1.0%    1.0%  noisy
montecarlo.analyse  10.10s   10.08s    0.3%   -1.2%  noisy
retest.load_sims     0.09s    0.10s   -6.8%    4.2%  noisy
crossmarket.paired   0.45s    0.44s    2.0%    3.0%  noisy

sin regresiones
```

Una línea por objetivo. `ahora` es el tiempo de esta medida, `antes` el de la anterior, `tiempo` el cambio **por unidad de trabajo** y `memoria` el cambio en el pico de memoria.

flag	obligatorio	qué hace
<code>--only</code>	no	mide sólo esos objetivos o esa área (<code>core</code> , <code>strategies</code> , <code>montecarlo.analyse</code>)
<code>--scaling</code>	no	vuelve a medir el techo de memoria de la máquina; tarda unos minutos
<code>--hotspots OBJETIVO</code>	no	en vez de medir todo, perfila uno y dice dónde se le va el tiempo
<code>--set CLAVE=VALOR</code>	no	cambia cualquier valor de <code>perf/config.yaml</code> sólo para esta corrida

El comando **sale con código 1 si algo ha empeorado**, así que puede ir en cron sin nadie delante.

El otro comando:

```
python3 -m perf.disk.report      # inventario de AlgoData, presupuestos y qué se puede borrar
```

```
$ python3 -m perf.disk.report
AlgoData: 2.35 GB en 49 ramas

2153.8 MB   4014 ficheros   1.0 d raw
2153.8 MB   4014 ficheros   1.0 d raw/XAUUSD
1599.8 MB   2121 ficheros   9.0 d raw/XAUUSD/WFM_Stability
310.3 MB    1517 ficheros   1.5 d raw/XAUUSD/MC_Trades
107.4 MB    104 ficheros    0.5 d logs
107.1 MB    81 ficheros     0.5 d logs/SQX
107.0 MB    34 ficheros     0.5 d logs/SQX/StrategyQuant
75.4 MB     91 ficheros     2.0 d raw/XAUUSD/MCR_All
65.9 MB     21 ficheros     1.0 d raw/XAUUSD/SPP_IS
43.5 MB     154 ficheros    6.5 d raw/XAUUSD/Retest_Markets_-_Family
43.2 MB     69 ficheros     9.0 d raw/XAUUSD/WFM
36.8 MB     4 ficheros     11.5 d bars

duplicados: 1 grupos, 12.8 MB recuperables
x2      12.8 MB  raw/XAUUSD/SPP_IS/2026-09-10/bars/bars_M30.csv | raw/XAUUSD/MC_Trades/2026-09-19/bars/bars_M30.csv
```

El presupuesto de disco

Desde el 2026-09-21, `perf.disk.report` también sale con código 1 si alguna rama de `AlgoData` se pasa de su presupuesto, o si el total se pasa del suyo. Igual que `perf.catalogue` con las regresiones: así puede ir en cron y creerte lo que dice.

```
AlgoData: 2.47 GB de 60 GB (4%) en 55 ramas

      ok      1.96 GB /   40 GB      5% raw
      ok      0.30 GB /    3 GB     10% bars
      ok      0.11 GB /    1 GB     11% logs
      ok      0.06 GB /    5 GB      1% reports
```

Los techos están en `perf/config.yaml`, en `disk.budget_gb`. Se dimensionaron con 2,3 GB en uso y dejando sitio para el estudio de variantes (~200 MB de operaciones por estrategia madre y mercado, así que el crecimiento va a caer en `raw/`).

Una rama `sin` presupuesto sale marcada `sin presupuesto`, no aprobada. Que aparezca una rama nueva y empiece a engordar es justo lo que esta tabla existe para que veas.

Qué se puede borrar

Y una última sección dice **qué se podría borrar y cuánto liberaría**:

```
candidatos a borrar: 6, 0.47 GB (nada se borra aquí)
  331.9 MB strategy_copies raw/XAUUSD/WFM_Stability/2026-09-10/wfm/strategies
  118.9 MB strategy_copies raw/XAUUSD/MC_Trades/2026-09-19/strategies
    0.9 MB superseded_export raw/XAUUSD/Retest_Markets_-_Family/2026-09-09
```

Cinco reglas, y **sólo una de ellas es un veredicto**:

regla	qué encuentra	cuánto te puedes fiar
<code>collected_variants</code>	databanks que el ledger del pipeline dice que ya se exportaron y se comprobó el hash	veredicto — está probado que el dato sobrevivió
<code>superseded_export</code>	un export fechado cuyo hermano más nuevo contiene todo lo que él tiene	candidato
<code>strategy_copies</code>	ficheros <code>.sqx</code> dentro de exports, que duplican lo que ya guarda el databank	candidato
<code>intermediates</code>	carpetas <code>raw/</code> y <code>trades/</code> de CSV al lado de un <code>trades.parquet</code> que ya las contiene (los exports posteriores al 23-09-2026 las borran solos)	candidato
<code>stale_branch</code>	nada escrito en 90 días	la más floja — viejo no es lo mismo que sobrante

⚠ **Nada del proyecto borra ninguna de estas cosas.** La lista sólo le pone un número a una decisión que sigue siendo tuya.

Un detalle que importa: `superseded_export` compara **contenidos**, no fechas. La primera versión comparaba fechas y propuso borrar `raw/XAUUSD/SPP_IS/2026-09-10` porque existe un `2026-09-19` — pero el del 19 sólo tiene `wfc_pairs/`, mientras que el del 10 tiene las tablas de permutaciones que lee todo el estudio SPP. Habría propuesto borrar la única copia de los datos de entrada. Ahora un export sólo cuenta como sustituido si el nuevo lo contiene **entero**.

(`perf/measure/runner.py` también se puede ejecutar, pero es interno: es el subprocesso que el arnés arranca para medir un objetivo aislado. No lo llares a mano.)

Qué salidas produce, y qué significan

Todo vive en `~/Desktop/AlgoData/profiling/` :

fichero	qué guarda
<code>history.csv</code>	una fila por medida, para siempre. Sólo crece : nunca se edita ni se borra
<code>scaling.csv</code>	las curvas de escalado de la máquina
<code>disk.csv</code> , <code>duplicates.csv</code> , <code>formats.csv</code>	el inventario del disco
<code>budgets.csv</code>	cada rama contra su presupuesto, una fila por corrida
<code>reclaimable.csv</code>	los candidatos a borrar, con sus bytes y su motivo

Las columnas de `history.csv`

Columna por columna:

- **segundos** — lo que tardó la medida mediana de las tres repeticiones.
- **microsegundos/unidad** — lo mismo dividido entre el trabajo hecho. **Ésta es la que decide**. Si el export crece de 500 a 3.000 operaciones, los segundos suben y esta columna no: el código no se ha vuelto más lento, hay más datos.
- **trabajo** — cuántas operaciones, filas, velas o ficheros procesó, con su unidad.
- **MB pico** — el pico de memoria residente **del árbol de procesos entero**, incluidos los trabajadores que el objetivo arranque. Es la cifra que decide si una corrida cabe en la máquina.
- **dispersión %** — cuánto se separaron entre sí las repeticiones de esta misma medida. Es el ruido del instrumento.
- **veredicto** — uno de cinco:

veredicto	qué significa
<code>first</code>	primera vez que se mide; no hay con qué comparar
<code>regression</code>	más del 15 % más caro por unidad, o más del 15 % más de memoria
<code>improvement</code>	más del 15 % más barato
<code>steady</code>	cambió poco, y ese poco es mayor que el ruido de la medida
<code>noisy</code>	cambió menos de lo que el instrumento puede distinguir. No es "igual": es "no lo sé"

El techo de la máquina (`scaling.csv`)

Las dos curvas son el mismo cálculo moviendo los mismos bytes. La única diferencia es si el dato cabe en la caché del procesador o hay que ir a buscarlo a la memoria principal.

Que la línea `cache` siga subiendo hasta 96 procesos y la `dram` se quede plana desde 16 significa que **los núcleos que sobran no están calculando: están esperando memoria**. La consecuencia práctica: en un módulo cuyos datos no caben en caché, pedir más procesos no lo hace más rápido — sólo gasta más RAM.

El disco

Las ramas más grandes son sólo las hojas: una carpeta que contiene a otra de la lista no aparece, porque su barra sería la suma de sus hijas otra vez. Los **duplicados son candidatos, no una sentencia**: dos ficheros que coinciden en tamaño y en 64 KB de cada extremo son casi seguro el mismo fichero, pero nada aquí lo confirma y **nada aquí borra nada**.

Dónde se le va el tiempo a un módulo

```
python3 -m perf.catalogue --hotspots core.sqx_stats
```

```
$ python3 -m perf.catalogue --hotspots core.sqx_stats
core.sqx_stats — 0.07 s profiled

TIME (own)
56.2%  0.039 s    100x  sqxstats.py:27(records)
 6.2%  0.004 s   19,020x ~:0()
 4.2%  0.003 s   15,762x ~:0()
 4.0%  0.003 s    6,248x pathlib.py:675(__lt__)
 2.9%  0.002 s   12,496x pathlib.py:654(_cparts)
 2.5%  0.002 s     20x  zipfile.py:1347(_RealGetContents)
 2.4%  0.002 s   140x  ~:0()
 1.8%  0.001 s     20x  ~:0()
 1.7%  0.001 s     1x  tracemalloc.py:477(_group_by)
 1.7%  0.001 s  11,620x ~:0()
 1.5%  0.001 s     21x  ~:0()
 1.4%  0.001 s   3,780x ~:0()

MEMORY (held at peak)
/usr/lib/python3.10/pathlib.py:969  0.0 MB
/usr/lib/python3.10/zipfile.py:1270  0.0 MB
/home/sergioguslw/Desktop/AlgoProject/core/sqxstats.py:48  0.0 MB
```

Arriba, las funciones ordenadas por **tiempo propio** — el gastado dentro de su propio cuerpo, sin contar a quién llaman. Abajo, las líneas que tenían más bytes vivos en el momento de mayor consumo.

Dos avisos:

- Los segundos de un perfilado son más largos que los reales, porque perfilar cuesta. **Lee el orden, no los segundos.**
- La lista de memoria sólo ve lo que reservó Python en ese proceso. Los buffers de numpy dentro de un trabajador no salen ahí: para eso está la columna `MB pico`, que muestrea el árbol entero.

Qué se puede tocar

Todo está en `perf/config.yaml`. Lo que de verdad cambia el resultado:

clave	qué hace
<code>harness.repeats</code>	repeticiones por medida. Menos repeticiones, más ruido y más veredictos <code>noisy</code>
<code>regression.wall_pct</code>	a partir de qué porcentaje una medida se llama regresión
<code>sample.n_sims</code>	el presupuesto de simulación del objetivo de Monte Carlo
<code>sample.files</code>	cuántos ficheros muerden los objetivos de parseo
<code>disk.stale_days</code>	a partir de cuántos días sin tocar una rama se marca como obsoleta

Cambiar `sample.*` rompe la comparación con las medidas anteriores, porque ya no es el mismo trabajo. Si lo cambias, la historia previa de ese objetivo deja de valer.

Las trampas que este módulo existe para evitar

- `ru_maxrss` miente cuando hay trabajadores. De los procesos hijos informa del más grande, no de la suma. Medido el 2026-09-20: `montecarlo.analyse` da 535 MB por `ru_maxrss` y **2.340 MB** muestreando el árbol. La segunda es la buena.
- El muestreo cada 50 ms puede perderse un pico más corto que eso. La cifra es un suelo.
- Una medida con la máquina ocupada no vale. Por eso cada fila guarda la carga del sistema y la memoria libre que había.
- Un commit `-dirty` no se puede reproducir. `history.csv` lo guarda tal cual para que se vea.

Lo que cuesta el workflow de punta a punta — medido 2026-09-24

Esto **no** sale de `perf.catalogue` y no está en `history.csv`, a propósito: el catálogo compara por unidad de trabajo y no toca SQX nunca, y esto es reloj de pared de una cadena que sí lo toca. Es una referencia de planificación, no una serie temporal.

La corrida medida: USDJPY H1, plantilla `emaCloseAbove`, proyecto `TestUSDJPY_Workflow_v1` en el custodio (48 núcleos, 80 g de heap), embudo 50 → 17 → 12 → 8 → 4. Los tiempos escalan con la población, así que están anotados con el tamaño al que se midieron.

En SQX — sacado del propio log (`Task finished in`)

paso	tarea	población	tiempo
6	CONSTRUCCION (build, 2008–2017)	→ 50 estrategias	16 s
7	OOS (retest <code>oos1</code> 2018–2022)	50	5,4 s
9	Retest Markets - Family (9 pares)	17 × 9	84 s
11	crossTF H1+H4	24 celdas	21 s
13	MCR 1 Bar	8 × 1.000 sims	24 s
13	MCR 5 Params	8 × 1.000	16 s
13	MCR 6 Exits	8 × 1.000	198 s
13	MCR 7 OHLC	8 × 1.000	695 s
13	MCR 8 Stress (3 perturbaciones, 15 años)	8 × 1.000	1.022 s
15	SPP IS	4 madres	79 s
15	SPP OOS	4 madres	42 s

El MC Retest es el 89 % del tiempo de SQX de toda la cadena: 1.955 s de 2.202. Y eso con CINCO tareas; con las siete que ahora se pueden configurar será más. Dentro del MC Retest, `OHLC` y `Stress` son el 88 %: perturbar el histórico y correr las tres perturbaciones juntas sobre 15 años.

En Python — `/usr/bin/time`, un núcleo salvo donde se diga

paso	comando	población	tiempo	RSS pico
8	<code>studies.screening.gate.harvest</code>	50 + 17	58 s	1,6 GB
8	<code>studies.screening.gate.report</code> (8 cribas + 17 monos a 2.000 sorteos)	17	2,4 s	380 MB
10	<code>crossmarket.report</code> a 2.000 sorteos	8 × 9	~13 min	—
10	<code>crossmarket.report</code> a 10.000 sorteos (el de <code>config.yaml</code>)	8 × 9	>50 min, no terminó	—
10.5	<code>variants.scale</code>	12 madres → 12 hermanas	0,7 s	125 MB
12	<code>crossTF.report</code>	24 celdas	29 s	580 MB
14	<code>retest.ingest</code>	36 corridas, 35.972 sims	10 s	3,2 GB
14	<code>retest.report</code>	4 estrategias	3,4 s	1,1 GB
16	<code>sppUltra.report</code>	4 perfiles, 49.226 filas	17 s	910 MB
16.5	<code>variants.make</code>	1 madre, 60 variantes	1,3 s	300 MB

Exportaciones, que arrancan y paran el conductor por su cuenta:

comando	población	tiempo	RSS pico
<code>export_metrics</code>	8 estrategias	15 s	20 MB
<code>export_spp</code>	4 perfiles	15 s	1,4 GB
<code>export_retest</code>	8 × 9 mercados, 66 k operaciones	17 s	1,9 GB

El impuesto que no aparece en ninguna tabla

Cada etapa exige parar el custodio para reescribir el `project.cfx` (regla dura 4) y arrancarlo otra vez. Medido:

	tiempo
<code>sqx-worker.sh start</code> (el script vuelve)	2,6 s
hasta que la CLI responde	21,5 s
<code>sqx-worker.sh stop</code> (con su sincronización de cierre)	14,7 s
ciclo completo	~39 s

En esta corrida se pagó **ocho veces: unos 5 minutos** sólo en abrir y cerrar. No es evitable hoy: `startOnlyTask` no corre nada en este install, así que una etapa por arranque es la única forma de parar entre pasos para cribar.

Los dos números que hay que tener en la cabeza

- `crossmarket.report` es el cuello de botella de Python, y por dos órdenes de magnitud: 13 minutos frente a segundos de todo lo demás. Un núcleo, 9 mercados × 10.000 sorteos por estrategia. Con una población de 100 en vez de 8 son horas. Es el primer candidato a paralelizar.
- `retest.ingest` pica 3,2 GB con 36 corridas. Escala con corridas × simulaciones, así que una población de 100 por las ocho tareas (800 corridas) pediría del orden de 70 GB si la proporción se mantiene. Antes de correr eso, medirlo.

Lo que NO está medido

Los pasos 17 (WFC), 18 (CSCV) y 19 (WFM) no se han corrido, porque gastan oos2. Tampoco el retest del lote de variantes del 16.5, que es el que los alimenta y el que se prevé más caro de todos: 240 variantes × 3 tramos × 9 mercados adicionales. Cualquier presupuesto de la cadena completa que salga de esta página está incompleto por ese lado, y no de poco.

Lo que cuesta NUESTRO Python, con 500 estrategias — medido 2026-09-24

La sección anterior mide la cadena entera con una población de juguete. Esta mide sólo la capa que hemos escrito nosotros, con 500 estrategias construidas y retesteadas a propósito para esto (proyecto PerfUSDJPY_Python_v1, USDJPY H1), y mide escalado, no un punto suelto.

Máquina: 96 núcleos, 125 GB. Todo lo que sigue usa UN núcleo.

La unidad correcta no es la estrategia, es la operación

Medido: las 8 estrategias de una misma corrida llevan entre 5.338 y 43.109 operaciones en los nueve mercados ajenos — un factor de 8. Un «segundos por estrategia» sobre esa población no dice nada. Por eso todo lo de abajo está normalizado por operación.

gate.* — el paso 8, cribar la población

N estrategias	studies.screening.gate.report	RSS
25	3,5 s	505 MB
50	5,2 s	545 MB
100	8,2 s	636 MB
250	15,9 s	819 MB
500	32,3 s	1,32 GB

Lineal y barato: unos 60 ms por estrategia de coste marginal, sobre un fijo de ~2 s. Incluye las 8 cribas y el mono de cada superviviente a 2.000 sorteos. 500 estrategias se criban en medio minuto.

studies.screening.gate.harvest, que es la mitad que conduce SQX: 112 s y 4,56 GB con 1.000 ficheros (500+500), contra 58 s y 1,6 GB con 67. El tiempo lo domina el arranque de la JVM; la memoria sí crece con la población y es el número a vigilar.

Dónde se le va el tiempo al gate (cProfile, N=500)

	s	%
total	33,4	100
monkey.mono	29,5	88
↳ simulate.nulls	15,1	45
↳ simulate.fixed	9,9	30
↳ ↳ calibrate.ctr	9,2	28
comparación de columnas object	4,2	13

Dos cosas concretas, y las dos son trabajo repetido, no trabajo necesario:

1. ☐ **El ATR se recalcula una vez por estrategia sobre las MISMAS barras.**

`engines/nulls/simulate.py:fixed()` llama a `calibrate.ATR(frame, ...)` y `frame` es idéntico en las 500 llamadas: $234 \text{ llamadas} \times 39 \text{ ms} = \mathbf{9,2 \text{ s de los 33}}$, y crece lineal con la población haciendo siempre la misma cuenta. Calcularlo una vez por (barras, periodo) lo deja en 39 ms totales.

2. ☐ **Se filtra la tabla entera de operaciones por identidad, una vez por estrategia**

(`studies/screening/gate/monkey.py:31`, `oos[oos["identity"] == name]`), y la columna es de tipo `object`: 4,2 s en 236 comparaciones de cadenas. Un `groupby` una sola vez lo elimina.

Juntas son **~40 % del paso 8** y ninguna cambia un número: es la misma cuenta hecha una vez.

crossmarket.report — el paso 10, y el cuello de botella real

Tres medidas limpias, con SQX parado y a 500 sorteos:

N	operaciones	tiempo	ms/operación	RSS
2	44.660	170,6 s	3,82	6,0 GB
4	133.240	507,4 s	3,81	6,8 GB
8	158.415	630,1 s	3,98	6,8 GB

La constante aguanta: 3,8–4,0 ms por operación. Y el coste crece con los sorteos, medido sobre una población fija de 122.045 operaciones: 4,19 ms/op a 500 sorteos y 5,03 a 1.000. Ajustando:

coste $\approx (3,36 + 0,00168 \times \text{sorteos})$ ms por operación, en un núcleo

Lo que significa para el export completo de las 500 estrategias, que lleva **12.010.976 operaciones** en diez mercados:

sorteos	un núcleo	con 90 núcleos
500	14 h	9 min
2.000	22 h	15 min
10.000 (el de su <code>config.yaml</code>)	67 h	45 min

⚠ El ajuste de los sorteos sale de dos puntos medidos **con SQX corriendo a la vez**, así que las tres cifras de la derecha son el orden de magnitud, no una promesa.

Dónde se le va el tiempo (cProfile, 8 × 9 mercados, 500 sorteos)

	tottime	cumtime
<code>simulate/metrics.py:paths</code>	64,8 s	184,1 s
↳ <code>_losing_run</code>	65,8 s	—
<code>verdict/stress.py:degraded</code>	65,4 s	65,5 s
<code>np.add.at</code> (<code>mechanics/equity.py:42</code>)	52,6 s	—
<code>model/holdfit.py:fit</code>	38,7 s	—
<code>cumsum</code> · <code>ufunc.reduce</code> (1,78 M llamadas)	28,5 · 28,1 s	—

☐ `np.add.at` → `np.bincount` medido, y **NO es el premio que parece**: 1,6x a 2,5x según el tamaño, no el 10x de la sabiduría popular. Verificado dando el mismo resultado.

□ `_losing_run` ya está vectorizado por caminos — el bucle recorre operaciones, no caminos, y cada paso es una operación vectorial. No es código ingenuo: es el coste inherente del barrido.

Conclusión: aquí no hay una micro-optimización que salve el día. El coste es `operaciones × sorteos × mercados × modelos` y está donde tiene que estar. Lo que sobra es que **corre en 1 de 96 núcleos**, y el bucle exterior sobre estrategias es independiente por construcción. Paralelizarlo es un cambio que no toca ni una fórmula y convierte 67 horas en 45 minutos. **Es la única optimización que importa de todo el proyecto.**

Los demás pasos de Python, a la escala a la que se pudieron medir

paso	comando	población	tiempo	RSS
10.5	<code>variants.scale</code>	12 → 12	0,7 s	125 MB
12	<code>crossTF.report</code>	24 celdas	29 s	580 MB
14	<code>retest.ingest</code>	36 corridas, 36 k sims	10 s	3,2 GB
14	<code>retest.report</code>	4	3,4 s	1,1 GB
16	<code>sppUltra.report</code>	4 perfiles, 49 k filas	17 s	910 MB
16.5	<code>variants.make</code>	1 madre, 60	1,3 s	300 MB

△ Estos seis **NO están medidos a 500**, y no por pereza: cada uno necesita trabajo de SQX proporcional a la población que no cabe en una sesión. El MC Retest son ~1.955 s por cada 8 estrategias, o sea **unas 34 horas para 500**; el SPP son ~20 s por madre, **2,8 h para 500**. Lo que sí se puede afirmar de ellos es la forma: `retest.ingest` escala con corridas × simulaciones y ya pica 3,2 GB con 36 corridas, así que 500 estrategias por 8 tareas (4.000 corridas) es el número que hay que medir antes de lanzarlo, no después.

Exportaciones, que son parte del coste de Python

comando	población	tiempo	RSS	escrito
<code>export_metrics</code>	8	15 s	20 MB	—
<code>export_retest</code>	8 × 9	17 s	1,9 GB	—
<code>export_retest</code>	500 × 9	154 s	6,3 GB	365 MB

Y un regalo de la medición: SQX es MÁS eficiente con lotes grandes

tarea	población	tiempo	por estrategia
build	50	16 s	0,32 s
build	500	30 s	0,06 s
retest OOS	50	5,4 s	0,11 s
retest OOS	500	13,9 s	0,03 s
crossmarket 9 mercados	17	84 s	4,9 s
crossmarket 9 mercados	500	293 s	0,59 s

Extrapolar linealmente desde una población pequeña **sobreestima SQX entre 4x y 8x**: paraleliza sobre los 96 núcleos y el coste fijo por tarea se diluye. Lo contrario que nuestro Python.

Lo mismo, ya paralelizado — medido 2026-09-25

Todo lo de la sección anterior corría en **un núcleo de 96**. Esta sección es la misma medida después de repartir el trabajo, sobre la misma población y los mismos ficheros. **Ninguna cifra del análisis cambia**: cada optimización se verificó comparando el resultado contra el de antes.

Qué se cambió, y por qué no cambia ningún número

cambio	dónde	por qué es exacto
el ATR se calcula una vez por fichero de barras, no una por estrategia	<code>engines/market/calibrate.py</code>	depende sólo de <code>(barras, ventana)</code> ; se cachea el mismo array
la tabla OOS se agrupa una vez por identidad, no se filtra una vez por estrategia	<code>studies/screening/gate/monkey.py</code>	<code>groupby</code> y la máscara booleana devuelven las mismas filas en el mismo orden
<code>stats.measure()</code> construye sólo la estadística pedida, y la puerta pide la única que lee	<code>engines/nulls/stats.py</code> , <code>studies/screening/gate/monkey.py</code>	el filtro estaba después del cálculo. <code>studies.readings.monkey.report</code> , que lee las cinco, cuesta lo mismo que antes
el bucle sobre estrategias se reparte entre los núcleos	<code>studies/screening/gate/monkey.py</code> , <code>studies/transfer/crossmarket/report.py</code>	las estrategias no comparten estado ni escriben nada
el estrés de ejecución se trocea en lotes de 500 corridas	<code>studies/transfer/crossmarket/simulate/stress.py</code>	los tres sorteos se siguen tomando enteros y en el mismo orden; sólo el precio va por lotes

Los cuatro usan `fork` : el padre lee el export entero y las barras **una vez** y los procesos hijos los heredan sin copiarlos. Mandárselos por `pickle` costaría más que el cálculo.

Paso 8 — `studies.screening.gate.report` , 500 estrategias

	antes	después
tiempo	31,1 s	5,2 s
RSS	1,30 GB	1,17 GB
CPU	143 %	2.437 %

5,9x. Verificado: el `scorecard.parquet` de 500 filas × 29 columnas sale **idéntico**, columna a columna, y sobreviven las mismas 229 estrategias.

El techo no es el reparto: de los 5,2 s, unos 2,5 s son leer la cosecha y correr las otras siete cribas, que ya eran baratas, y 0,48 s la maquinaria de reparto. El mono en serie baja de **29,5 s a 8,54 s** en tres pasos: el ATR cacheado y el `groupby` lo dejan en 15,7 s, y pedir una estadística en vez de cinco lo baja a 8,54 s.

Dentro del mono, y la dispersión que la media esconde

Reparto del mono ya optimizado, agregado sobre las 234 estrategias que llegan a él:

fase	%
precio de cada operación (<code>barrier.pnl</code>)	29,1
numpy suelto: <code>repeat / tile</code> , <code>reduce</code> , <code>std</code> , <code>mean</code>	28,9
sortear las entradas aleatorias (<code>model._place</code>)	27,7
barrido de barreras	13,2
reconciliar · fontanería · estadísticas · rejilla · p	1,1

□ **Las estadísticas eran el 46 % de este paso y cuatro de las cinco se tiraban.** `stats.measure()` calculaba `net`, `sharpe`, `pf`, `retdd`, `dd` para las 2.000 corridas de cada estrategia y la puerta lee `sharpe`. Medido sobre 2.000×570: **22,45 ms las cinco, 1,75 ms sólo `sharpe`** — y quien pide las cinco no paga nada por el cambio.

Y una por una, con 2.000 sorteos cada una:

	mín	p25	mediana	p75	máx	media
ms por estrategia	3,2	14,0	29,2	48,0	268,3	36,5
operaciones OOS	39	497	879	1.289	6.192	1.001
µs por operación	25,3	29,4	34,2	40,5	84,1	36,0

⚠ **La más cara cuesta 83,4x la más barata.** Presupuesta por operación, no por estrategia: la correlación entre tiempo y operaciones es **$r = 0,982$** .

Paso 10 — `crossmarket.report`

Con 8 estrategias × 9 mercados a 500 sorteos (158.415 operaciones), que es la medida que la sección anterior dejó hecha en un núcleo:

	antes	después
tiempo	623,5 s	156,1 s
CPU	102 %	375 %

4,0x con sólo 7 procesos, porque siete estrategias en siete procesos duran lo que la más larga de las siete — y ese lote va de 50 a 44.771 operaciones. `verdict.csv` sale **idéntico byte a byte**.

Y la población entera, que es el número que importa: 499 estrategias × 9 mercados a 500 sorteos, 72 procesos, 2.285 s — 38,1 minutos. Contra 12,8 h en un núcleo.

□ **El coste por operación es una constante de verdad, comprobado con un control.** Ocho estrategias del medio de la población, con casi las mismas operaciones que las ocho primeras, en un solo proceso:

lote	operaciones	segundos, 1 proceso	ms/operación
las 8 primeras	158.415	623,50	3,94
8 del medio	157.571	593,94	3,77

De ahí el «antes» de las 499: $12.010.976 \times 3,85 \text{ ms} = 46.278 \text{ s} \approx \mathbf{12,8 \text{ h}}$.

El reparto del paso 10 se mueve con el tamaño de la estrategia

Perfiladas tres del mismo lote, con un factor de 8 en operaciones. Dar una sola habría engañado:

fase	5.563 ops	8.988 ops	44.771 ops
numpy suelto (percentiles, <code>reduce</code> , <code>cumsum</code> , <code>add.at</code>)	54,0 %	46,8 %	31,6 %
estadísticas acumuladas de cada camino	17,9 %	20,9 %	33,3 %
sortear: los 4 modelos de colocación	9,4 %	10,2 %	13,9 %
estrés de ejecución	6,6 %	7,5 %	9,7 %
test pareado contra estar largo	4,9 %	6,4 %	1,4 %
preparar el mercado y reconciliar el fill	3,0 %	3,4 %	4,4 %
todo lo demás	4,1 %	3,9 %	2,7 %
ms por operación	5,09	4,46	3,64

Abierto por función, **nada pasa del 11 %** (`_losing_run` 11,1 %, `np.partition` 8,6 %, `metrics.paths` 6,7 %, `np.add.at` 6,4 %, `stress.degraded` 6,2 %, `holdfit.fit` 6,1 %). Aquí no hay bala de plata: el premio fue el reparto.

Lo que cuesta la maquinaria de paralelizar

Padre de 1,02 GB, 234 tareas, tarea vacía para medir sólo la fontanería:

procesos	crear el pool (<code>fork</code>)	repartir y recoger 234 tareas
16	0,00 s	0,11 s
48	0,00 s	0,26 s
96	0,00 s	0,48 s

`fork` es instantáneo porque no copia, y cada resultado del paso 8 son **45 bytes**. La comunicación no es el cuello de botella, precisamente porque no se manda nada grande.

La memoria era el límite de verdad, y estaba en un sitio concreto

□ `stress.simulate` construía la matriz entera de **25.000 corridas × operaciones del mercado** de una vez, y tres de los arrays eran `float64` donde los valores son booleanos:

operaciones del mercado	pico antes	pico después
684	816 MB	140 MB
607	724 MB	137 MB
563	672 MB	136 MB

⚠ **Esto no era un lujo.** El primer intento de correr las 500 estrategias con 96 procesos llegó a **94,5 GB de los 125** y hubo que abortarlo; un segundo intento con 48 procesos llegó a **89 GB** y el núcleo mató la ventana de VSCode. Con el troceado, **72 procesos ocupan 22 GB**.

La regla: `--workers` es el mando que cambia RAM por reloj. Cuenta **~0,35 GB por proceso** más lo que ocupe el export en el padre, y deja margen: la máquina también está siendo usada.

Cuántos procesos conviene usar — y no son 96

96 estrategias (2,43 M operaciones), 500 sorteos, el mismo trabajo a cinco tamaños de reparto. La referencia de un núcleo son 9.370 s por la constante de 3,85 ms/operación:

procesos	segundos	CPU	aceleración	eficiencia	horas-CPU
12	1.041,4	851 %	9,0x	75 %	2,5
24	759,2	1.324 %	12,3x	51 %	2,8
48	695,1	2.317 %	13,5x	28 %	4,5
72	673,7	3.293 %	13,9x	19 %	6,2
96	660,4	3.801 %	14,2x	15 %	7,0

De 24 a 96 procesos se gana un 13 % de reloj y se gastan 2,5x más horas-CPU.

□ **Y la causa no es Amdahl.** El tramo en serie —leer el export y las barras antes de lanzar a nadie— está medido y es **0,83 s de 695** con 96 estrategias, **2,38 s** con las 499. Eso autorizaría 800x, no 14x.

□ **La causa es el tamaño de la tarea.** Una tarea es una estrategia, y dentro del mismo lote la mayor lleva **117.612 operaciones (453 s ella sola)** y la menor **12**: un factor de 10.000. A partir de 24 procesos el reloj no lo manda el reparto, lo manda la estrategia más larga:

procesos	suelo teórico	medido	quién manda
12	781 s	1.041 s	el reparto
24	453 s	759 s	la más larga
96	453 s	660 s	la más larga

El hueco entre 453 y 660 sí es contención, pero aunque se borrara no se bajaría de 453 s. **El siguiente paso real es repartir por (estrategia, mercado)** : los nueve mercados son independientes dentro de `analyse_market` y eso divide la tarea más larga por ~9.

Recomendación: para el paso 10, **24-32 procesos**. Dan el 87 % del reloj de 96 con una fracción de la memoria, y la memoria es lo que mató la ventana de VSCode.

Lo que se midió y se decidió NO hacer

- `_losing_run` recorriendo filas contiguas en vez de columnas con salto: 4,5x sobre la matriz de 25.000×600, pero **sólo 1,5x** sobre los lotes de 500 en los que ahora se trabaja, y es ~13 % del tiempo. Resultado idéntico verificado. Queda medido por si el troceado cambia de tamaño.
- `np.add.at` → `np.bincount` : 1,6x–2,5x, ya medido el 2026-09-24. No cambia el orden de magnitud.

Segunda ronda: numba y balanceo de carga — medido 2026-09-25

Misma máquina, mismos ficheros y mismas órdenes que arriba, con la base **re-medida el mismo día** desde el commit anterior (`f1bf407`) en una copia aparte del código. `PYTHONHASHSEED=0` , un hilo de BLAS. Datos crudos y scripts: `AlgoData/reports/perf-optim-2026-09-25/` .

Qué se cambió

#	cambio	dónde
1	kernel numba que valora cada run y calcula sus estadísticas en una pasada, sin matrices intermedias	<code>engines/nulls/kernel.py</code> , <code>engines/nulls/placement/kernel.py</code>
2	barrido de barreras que se para en el primer toque	<code>engines/nulls/kernel.py:touched</code>
3	balanceo de carga: lo más caro primero (LPT), un hilo de BLAS por proceso	<code>core/fanout.py</code>
4	el paso 10 reparte por (estrategia, mercado) y calcula solo lo que publica el <code>verdict.csv</code>	<code>studies/transfer/crossmarket/report.py</code> , <code>market_run.verdict_row</code>
5	<code>studies.readings.monkey.report</code> en paralelo y leyendo el export una vez , no una por estrategia	<code>studies/readings/monkey/report.py</code>
6	semilla estable por (estrategia, peldaño, bloque) y bloque medido en trades	<code>engines/nulls/simulate.py</code>
7	la cosecha exporta IS y OOS en un ciclo del conductor y un <code>orderstocsv</code>	<code>studies/screening/gate/collect.py</code>

El resultado

tarea	antes	después	factor	RAM del sistema, antes → después
<code>studies.readings.monkey.report</code> , 757 estrategias × 4 peldaños × 2.500	569,0 s	11,1 s	51x	1,0 → 5,3 GB
<code>crossmarket</code> 8 × 9, 7 procesos	155,7 s	12,9 s	12x	8,6 → 1,5 GB
<code>crossmarket</code> 96 × 9, 24 procesos	755,4 s	52,2 s	14x	20,8 → 7,3 GB
<code>crossmarket</code> 96 × 9, 96 procesos	674,9 s	59,4 s	11x	37,4 → 21,1 GB
<code>crossmarket</code> 499 × 9 , 72 procesos	2.285 s (38 min)	255 s (4 min)	9x	53,8 → 32,7 GB
<code>studies.screening.gate.report</code> , 500 estrategias	5,6 s	4,9 s	1,15x	5,3 → 1,1 GB
<code>studies.screening.gate.harvest</code> , 500 + 500 (export)	114,1 s	81,5 s	1,4x	4,4 → 4,4 GB
<code>export_retest</code> , 499 × 9 (export)	160,4 s	160,0 s	1,0x	6,0 → 5,8 GB

Cuántos procesos, ahora

procesos	96 × 9, después
12	90,8 s
24	52,2 s
48	44,6 s
72	51,7 s
96	59,4 s

Recomendación nueva: 48 procesos, uno por núcleo físico. Por encima el hyperthreading y la caché compartida lo hacen **más lento**, no igual, y además gasta más memoria.

Por qué los números son los mismos

- `crossmarket` : el `verdict.csv` sale **idéntico byte a byte** al de antes en 8×9 y en 96×9 , con 12, 24, 48, 72 y 96 procesos. El 499×9 no se pudo comparar: su salida anterior se sobrescribió.
- Los kernels se compararon contra el código numpy con los mismos sorteos: drawdown, curvas, racha y trades **idénticos bit a bit**; net, Sharpe y PF a $\leq 5,5 \cdot 10^{-11}$ (numpy suma por pares, el kernel en orden).
- `nulls` y el mono de la puerta **cambian de monos**, a propósito (semilla nueva). 15.140 p de `studies.readings.monkey.report` : correlación 0,991 con los de antes, y solo el 0,23 % se aleja más de 3 errores Monte Carlo (se espera $\sim 0,3$ %). La puerta: las cinco cribas deterministas idénticas; supervivientes 229 $\rightarrow$ 228, una estrategia en el umbral.
- La cosecha: métricas y equity idénticas; operaciones idénticas en valores, con `Sample type` ahora categórica en vez de texto.

Lo que se midió y NO funcionó

- ☐ **Parsear los CSV del export con 16 hilos es más lento**: `export_retest` 179,5 s contra 160,4 s en serie, misma salida. Lo que se hace con cada fichero después de `read_csv` retiene el GIL. Se deshizo.
- En el export manda SQX: el `orderstocsv` y la carga del databank. La cosecha solo ganó lo que cuesta un arranque de JVM; `export_retest` ya tenía uno solo y no ganó nada.

Dónde está el cuello de botella ahora, y qué se estropea al doblar procesos

☐ Medido el 2026-09-25 cronometrando cada fase de cada tarea (estrategia, mercado) de 96×9 , a 24, 48 y 96 procesos (`AlgoData/reports/perf-optim-2026-09-25/phases_*.parquet`):

- **No es el reparto**: los procesos están ocupados el 99 % del reloj en los tres casos, y la última tarea dura menos de 3 s. La parte en serie (leer el export y compilar) es 1,5 s.
- **Cada tarea se vuelve más lenta cuantas más corren a la vez**, y sin esperar a nada (CPU/reloj = 1,00). El mismo trabajo cuesta 1.155 s de proceso con 24, 1.940 con 48 y **5.219 con 96**. De 48 a 96 dos procesos comparten núcleo físico. De 24 a 48 ya se pierde un 68 %: se comparten L3 y memoria, y ☐ probablemente baja el turbo.
- **Lo que más se estropea de 48 a 96**: el kernel numba de los nulos, **6,2 veces** más lento, y el estrés de slippage, 5,6 veces. Son las fases que leen los arrays de precios a saltos. El resto empeora entre 2,2 y 3,4 veces.
- **Lo que más pesa, con 48 procesos**: el **test pareado** (`paired.run`), el **42 %** del tiempo de cada tarea; la exposición (bootstrap), el 15 %; la reconciliación (`backtest.setting`), el 12 %; el bootstrap de PF y esperanza, el 10 %. Los nulos enteros (sortear, kernel, diagnósticos) ya son sólo el 12 %. **El siguiente objetivo es `paired.run`**, no los nulos.

Tercera ronda: todo lo demás — medido 2026-09-25

Todos los procesos de Python del proyecto que no se habían optimizado, con la base re-medida el mismo día desde el commit anterior en una copia aparte del código. **Memoria = PSS del árbol de procesos**

(`/proc/<pid>/smaps_rollback`), no RSS: con `fork`, el RSS cuenta una vez por hijo las páginas que comparten (`knowhow/perf/`). Datos crudos, scripts y validaciones: `AlgoData/profiling/bench-2026-09-25/`.

proceso	antes	después	factor	memoria, antes → después	qué se cambió
Monte Carlo, 36 estrategias × 20.000 caminos	477 s	27 s	18x	1,7 → 2,7 GB	cada estrategia en su proceso, LPT; kernel numba de los estadísticos sin la matriz reunida
<code>studies.screening.filters.report</code> , 10.000 estrategias	38,7 s	3,2 s	12x	0,4 → 1,5 GB	los 420 filtros candidatos, repartidos
CSCV (<code>pbo</code>), 962 variantes	30,8 s	9,5 s	3,3x	igual	vecinos de la rejilla calculados una vez; las tres reglas a la vez
crossTF, 48 celdas	29,2 s	5,7 s	5,1x	igual	caché de los YAML de <code>assets/</code> (54 de 60 s eran parsearlos)
<code>sqx.variants.make</code> , 5.000 variantes	11,2 s	2,1 s	5,3x	igual	fabricar y releer los <code>.sqx</code> en paralelo
<code>studies.readings.monkey.report</code> (tras la ronda 2)	11,1 s	3,6 s	3,1x	—	la misma caché de YAML
<code>retest.ingest</code> , 5 × 8 tareas	7,9 s	1,9 s	4,0x	2,9 → 1,9 GB	cada proceso escribe su P&L; 16 procesos
<code>sppUltra</code>	7,0 s	1,9 s	3,7x	igual	la prueba de inertes, vectorizada
<code>retest.report</code>	5,3 s	2,0 s	2,6x	0,4 → 0,85 GB	una estrategia por proceso
<code>entryQuality</code>	2,5 s	2,1 s	1,2x	1,3 → 0,47 GB	lee sólo la columna M1 que usa
<code>index_sqx</code> , 19.328 <code>.sqx</code>	2,2 s	1,65 s	1,3x	igual	abre cada zip una vez, no dos (~28 % CPU)
los diez restantes (WFM, WFC, parameterCloud, profitShape, exposure, decay, is_oos, tasks.nulls, ledger, variants.scale)	0,4–1,9 s	igual	1x	igual	su tiempo es importar pandas y scipy (~0,7 s)

Suma de los 20: 623 s → 67 s.

Por qué los números son los mismos

- **Idénticos byte a byte o celda a celda:** los ficheros de `sppUltra` y `retest.report` (hoy `spp` y `mcRetest`), el `cscv.json`, las 48 celdas de `crossTF`, el `improvement.md` de `filters` (hoy `filters/filters.md`), el `nulls.csv`, el índice de `.sqx`, el manifiesto de `variants.make` y el contenido de sus 5.000 `.sqx`, la salida de `entryQuality`, y las cinco tablas de la ingesta, incluidas las **35.125.203 filas de P&L**.
- **Kernel de Monte Carlo** contra `metrics.paths` con los mismos índices, en los 5 modelos de sorteo y los 4 de estrés: $\leq 2,5 \cdot 10^{-15}$ relativo, drawdown y racha idénticos.
- **Monte Carlo es aleatorio sin semilla**, así que se validó contra su propio ruido: dos corridas nuevas y una base dan el **mismo tier en las 36 estrategias**, y la base se separa de la nueva lo mismo que la nueva de sí misma.

Lo que se encontró por el camino

- □ **@njit por defecto lanza ZeroDivisionError** donde numpy da `nan`. Los tres kernels llevan ya `error_model="numpy"`, también los de la ronda 2.

- `retest.ingest` no gastaba memoria en el padre, sino en los trabajadores (~85 MB cada uno). Con 40 va en 1,4 s y 3,4 GB; con 16, en 1,9 s y 1,9 GB; con 8, en 2,7 s y 1,1 GB. Se dejó en 16.
- `sqx.variants.equity` y `collect` no se pudieron medir: no hay en disco ninguna carpeta de variantes con el formato de tres tramos actual.

El paso 16.5 de punta a punta — medido 2026-09-25

Tres madres de USDJPY H1, 5.000 variantes cada una, retesteadas en 3 tramos × 10 mercados en el custodio. Tabla completa en `09-optimizacion.pdf` > `19-wfc`. Lo que cambió en la cosecha, medido sobre la misma escala y validado contra la lógica original en 200 ficheros de cada tramo:

fase	antes (madre 1)	después (madres 2 y 3)
<code>equity</code>	385 s, 31,8 GB	83–88 s, 3,7–4,4 GB
<code>collect</code>	196 s	17 s
vaciar el custodio	206 s arrancando SQX, que carga las 20.000 estrategias (JVM 47 GB) solo para borrarlas	3–4 s borrando con SQX parado
Python por madre	~13 min	~1,8 min

Lo que manda es SQX: 63–71 min de retest por madre, con el JVM pegado a su techo de 80 GB.